Threaded Programming

Lecture 6: Further topics in OpenMP

Overview

- Nested loops
- Orphaned constructs
- Thread-private globals
- User defined reductions
- Timing routines

Nested loops

- For perfectly nested rectangular loops we can parallelise multiple loops in the nest with the `collapse` clause:

```
#pragma omp parallel for collapse(2)
for (int i=0; i<N; i++) {
  for (int j=0; j<M; j++) {
         .....
  }
}
```

- Argument is number of loops to collapse starting from the outside
- Will form a single loop of length NxM and then parallelise and schedule that.
- Useful if N is O(no. of threads) so parallelising the outer loop may not have good load balance

Orphaned directives

- Directives can be present in functions called from inside parallel regions
- Example:

```
#pragma omp parallel
{
   fred();
}

void fred() {
#pragma omp for
   for (int i=0; i<N; i++) {
      a[i] += 23.5;
   }
}
```

Orphaned directives (cont)

- This is very useful, as it allows a modular programming style….
- But it can also be rather confusing if the call tree is complicated (what happens if `fred` is also called from outside a parallel region? - the worksharing loop is all executed by the master thread)
- There are some extra rules about data scope attributes….

Data scoping rules

When we call a subroutine from inside a parallel region:

- Variables in the argument list passed by reference inherit their data scope attribute from the calling routine.
- Global variables are shared, unless declared `threadprivate` (see later).
- `static` local variables are shared.
- All other local variables are private.

Thread private global variables

- It can be convenient for each thread to have its own copy of variables with global scope
- Outside parallel regions and in `masked` directives, accesses to these variables refer to the inital thread's copy.

C/C++: `#pragma omp threadprivate (`*list*`)`

This directive must be at file or namespace scope, after all declarations of variables in *list* and before any references to variables in *list*. See standard document for other restrictions.

The `copyin` clause allows the values of the initial thread's `threadprivate` data to be copied to all other threads at the start of a parallel region.

User defined reductions

- Cannot use built-in reduction operators on objects or structures.

- Use **`declare reduction`** directive to define new reduction operators
- New operators can then be used in reduction clause.

```
#pragma omp declare reduction (reduction-identifier :
typename-list : combiner) [identity(identity-expr)]
```

- `reduction-identifier` gives a name to the operator
  - Can be overloaded for different types
  - Can be redefined in inner scopes
- `typename-list` is a list of types to which it applies
- `combiner` expression specifies how to combine values
- `identity` can specify the identity value of the operator

  Can be an expression or a brace initializer

Example

```
#pragma omp declare reduction (merge : std::vector<int>
: omp_out.insert(omp_out.end(), omp_in.begin(), omp_in.end()))
```

- Private copies created for a reduction are initialized to the identity that was specified for the operator and type
  - Default identity defined if identity clause not present
- Compiler uses combiner to combine private copies
- `omp_out` refers to private copy that holds combined values
- `omp_in` refers to the other private copy
- Can now use `merge` as a reduction operator.

Timing routines

OpenMP supports a portable timer:

- return current wall clock time (relative to arbitrary origin) with:

  ```
  double omp_get_wtime(void);
  ```

- no guarantee about clock precision but it can be queried with:

  ```
  double omp_get_wtick(void);
  ```

Usage:

```
double starttime, time;
starttime = omp_get_wtime();
   ......(work to be timed)
time = omp_get_wtime()- starttime;
```

Timers are local to a thread: must make both calls on the same thread.

Exercise

Molecular dynamics again

- Aim: use of orphaned directives.
- Modify the molecular dynamics code so by placing a parallel region directive around the iteration loop in the main program, and making *all* code within this sequential except for the forces loop.
- Don't expect this to change the performance much: the idea is to see how it affects data-attribute scoping
- Be careful about harmless-looking race conditions!

Fortran example - orphaned directives

```
!$OMP PARALLEL
      call fred()
!$OMP END PARALLEL

      subroutine fred()
!$OMP DO
      do i = 1,n
         a(i) = a(i) + 23.5
      end do
      return
      end
```

Fortran data scoping rules

When we call a subroutine/function from inside a parallel region:

- Variables in the argument list inherit their data scope attribute from the calling routine.
- COMMON blocks or module variables in Fortran are shared, unless declared **`THREADPRIVATE`** (see below).
- **`SAVE`** variables in Fortran are shared.
- All other local variables are private.

`!$OMP THREADPRIVATE (`*list*`)`

where list contains named common blocks (enclosed in slashes), module variables and **`SAVE`** variables..

- This directive must come after all the declarations for the COMMON blocks or variables.

Fortran example - using timers

```
DOUBLE PRECISION STARTTIME, TIME

STARTTIME = OMP_GET_WTIME()
......(work to be timed)
TIME = OMP_GET_WTIME()- STARTTIME
```

Reusing this material